

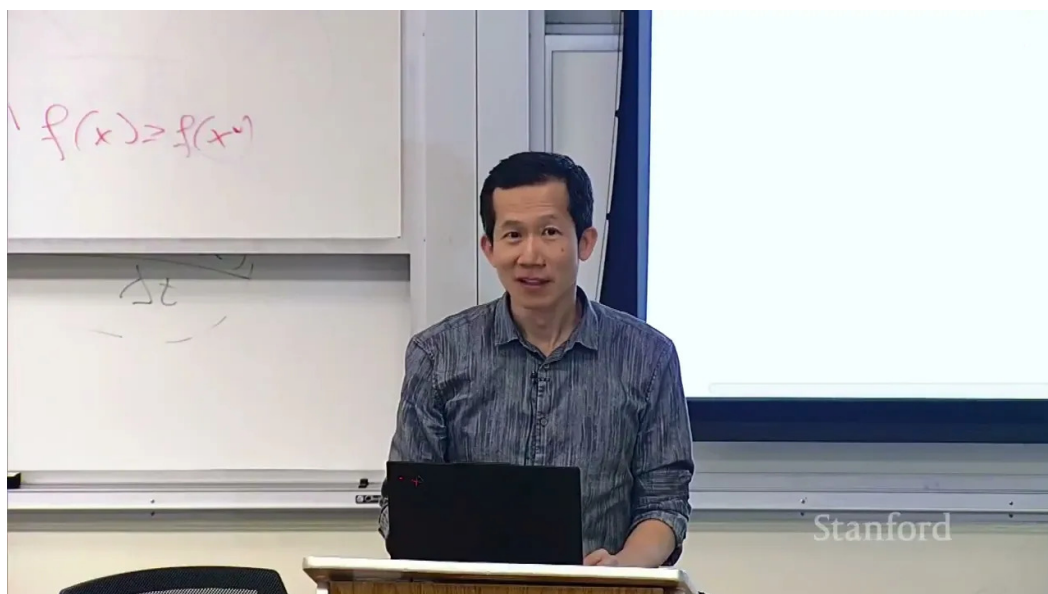
课程笔记

从零构建语言模型：课程全景与分词

Stanford CS336 Spring 2026 · Lecture 1 高密度中文讲义

Percy Liang (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online

发布日期: 2026-04-14

视频时长: 01:19:21

视频链接: <https://www.youtube.com/watch?v=JuoVZkPBiKk>

课程主页: <https://cs336.stanford.edu/>

官方可执行课件: `lecture_01.py`

目录

1	为什么还要“从零构建”语言模型	3
1.1	抽象提升生产率，也可能切断因果理解	3
1.2	工业化之后，公开课堂能学什么	4
1.3	本章小结	6
2	效率主线与“苦涩教训”	6
2.1	同一套效率问题贯穿全栈	7
2.2	本章小结	7
3	语言模型发展史：接口在变，基本机制仍在	7
3.1	从统计模型到神经基础模型	7
3.2	开放生态为何重要	8
3.3	语言模型接口的四次变化	9
3.4	本章小结	10
4	课程地图一：从 Tokenizer 到训练循环	10
4.1	Tokenization：模型究竟在哪些“原子”上计算	10
4.2	模型架构：从原始 Transformer 到现代组件	11
4.3	训练配方：参数如何被可靠地设置	12
4.4	本章小结	12
5	课程地图二：系统、并行与推理	12
5.1	资源核算与 Roofline 视角	13
5.2	Kernel：把数学表达式组织成硬件友好程序	13
5.3	并行：通信是跨 GPU 的数据搬运	14
5.4	推理：Prefill 与 Decode 是两种工作负载	15
5.5	本章小结	15
6	课程地图三：缩放定律与可预测训练	15
6.1	用小实验优化大训练配方	16
6.2	等算力曲线：更大模型还是更多 token	16
6.3	本章小结	17
7	课程地图四：评测、数据与对齐	17
7.1	评测服务于两类不同目的	17

7.2	数据不是“从天上掉下来”的	17
7.3	数据处理是一条有损决策链	18
7.4	对齐：当评价比生成更容易	19
7.5	一条贯穿五个作业的主线	19
7.6	本章小结	20
8	Tokenization 基础：可逆表示与计算预算	20
8.1	现实 Tokenizer 的几个反直觉现象	21
8.2	压缩率与词表大小	21
8.3	本章小结	22
9	三种直接方案为什么都不理想	22
9.1	字符级：语义直观，但词表大且长尾严重	22
9.2	字节级：词表固定，却把序列拉得过长	23
9.3	词级：压缩好，却无法封闭词表	24
9.4	本章小结	24
10	Byte Pair Encoding：从字节学习可变粒度词表	24
10.1	基本直觉	24
10.2	最小实现与复杂度	26
10.3	用合并表编码新文本	26
10.4	Tokenizer-free 的长期方向	27
10.5	本章小结	28
11	总结与延伸	28
11.1	讲者总结	28
11.2	结构化提炼	28
11.3	建议练习	29
11.4	拓展阅读	29
11.5	本章小结	29

1 为什么还要“从零构建”语言模型

CS336 的出发点不是否定 API、预训练权重或高层框架，而是指出：语言模型的抽象层仍然会**泄漏**。当研究者只会调用闭源模型时，一旦遇到上下文、推理效率、数据、稳定性或训练目标层面的限制，就很难判断失败来自哪里，更无法改造底层机制。课程因此采用“通过构建获得理解”的路线：亲手完成 tokenizer、Transformer、优化器、训练循环、GPU kernel、分布式训练、数据处理与对齐。

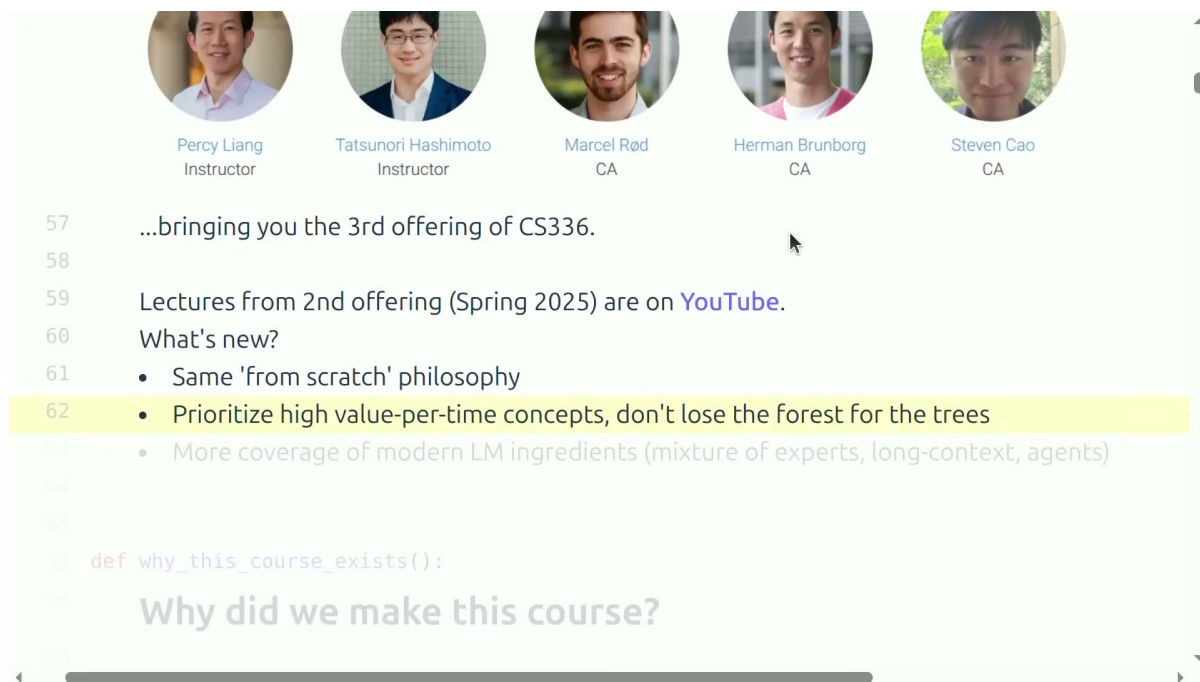


图 1: 2026 年第三次开课：延续从零构建理念，同时强化现代架构、长上下文与智能体内容。¹

1.1 抽象提升生产率，也可能切断因果理解

讲者用研究工作流的变化说明断层：早期研究者训练自己的模型，随后转向下载并微调 BERT，再后来主要通过提示 GPT、Claude 或 Gemini 完成任务。抽象层越高，试验速度越快；但语言模型不像编程语言或操作系统那样形成稳定边界，模型能力、推理接口、数据和系统实现仍在共同变化。

¹视频画面时间区间：00:02:45–00:03:00。

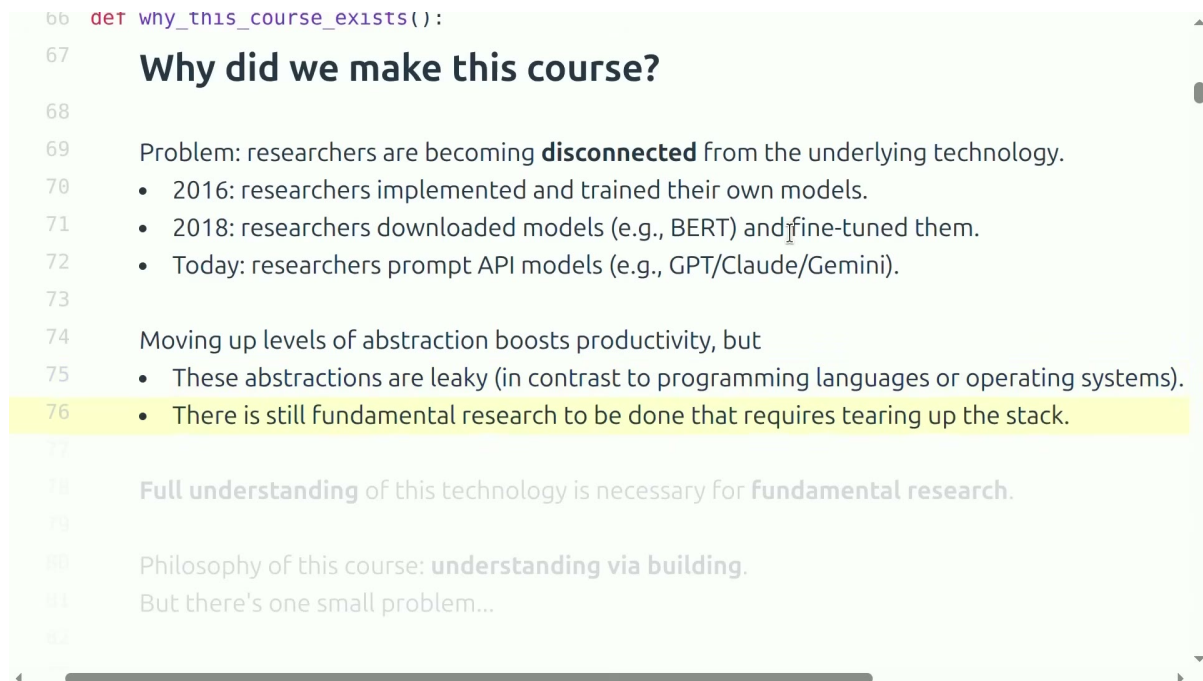


图 2: 课程存在的理由: 研究者逐渐与底层技术脱节, 而当前抽象仍不足以屏蔽底层差异。²

课程真正要培养的能力

目标不是记住一套 2026 年的最佳实践, 而是获得定位问题的能力: 看到损失异常时能追到数值尺度, 看到吞吐低时能追到数据搬运, 看到 token 成本高时能追到 tokenizer, 看到长上下文失效时能区分建模、训练分布和服务系统。

1.2 工业化之后, 公开课堂能学什么

前沿模型已经成为资本和基础设施密集型工程: 训练预算达到上亿美元, 集群规模达到十万 GPU 量级, 而闭源技术报告常省略数据、架构、训练和失败细节。小模型实验又不总能代表大模型: 注意力与 MLP 的 FLOPs 比例会随形状变化, 有些行为只在临界规模之后出现。

²视频画面时间区间: 00:04:15–00:04:30。

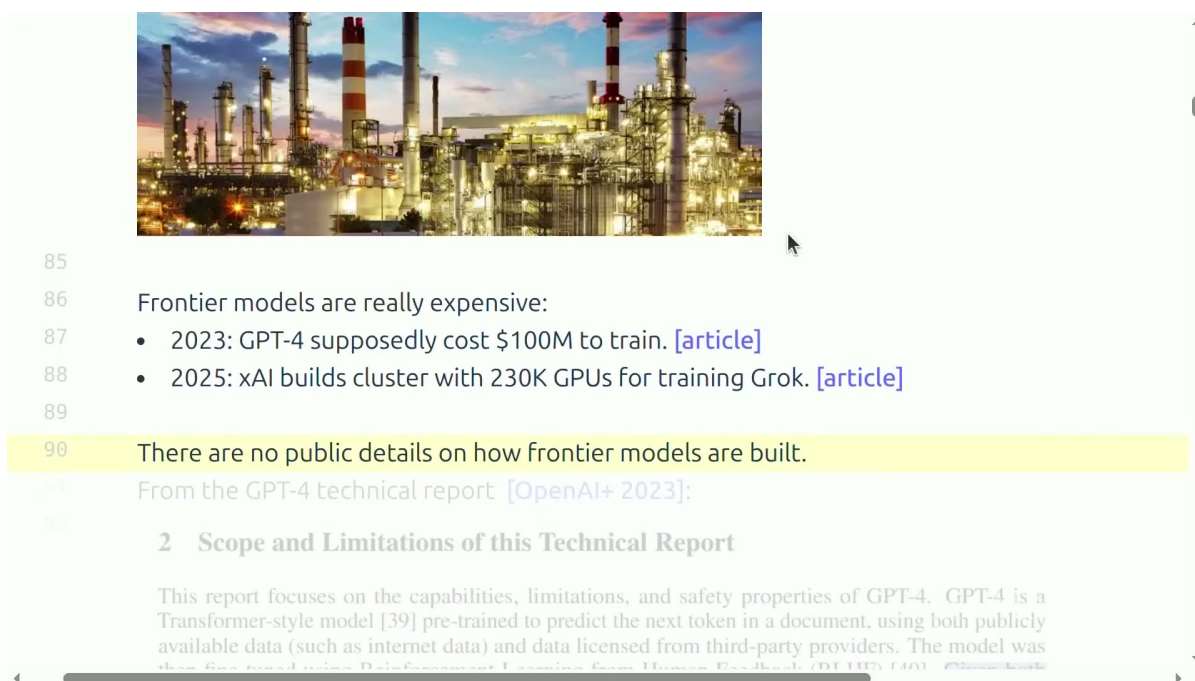


图 3: 前沿模型工业化: 训练成本和集群规模急剧上升, 但关键建模细节并未公开。³

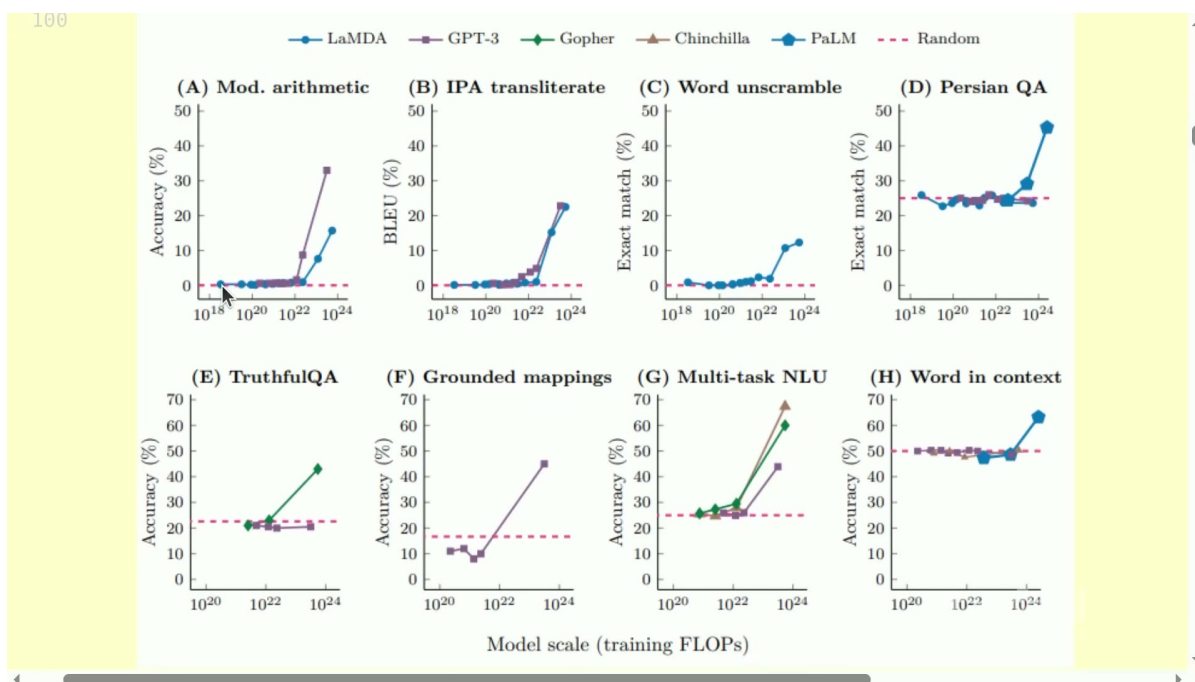


图 4: 规模变化可能改变能力曲线; 在小规模上看不到的现象, 不能简单外推为不存在。⁴

课程把知识分成三层:

- **Mechanics (机制):** Transformer、并行、kernel 等怎样工作;
- **Mindset (思维方式):** 认真核算资源、持续 profiling、把扩展性放在中心;
- **Intuitions (经验直觉):** 哪些数据和建模选择更准。

³视频画面时间区间: 00:05:15–00:05:30。

⁴视频画面时间区间: 00:07:00–00:07:15。

机制和思维方式通常能跨规模迁移；经验直觉却可能随数据、硬件和规模改变。课程承诺前两者，只对第三者给出有边界的经验。

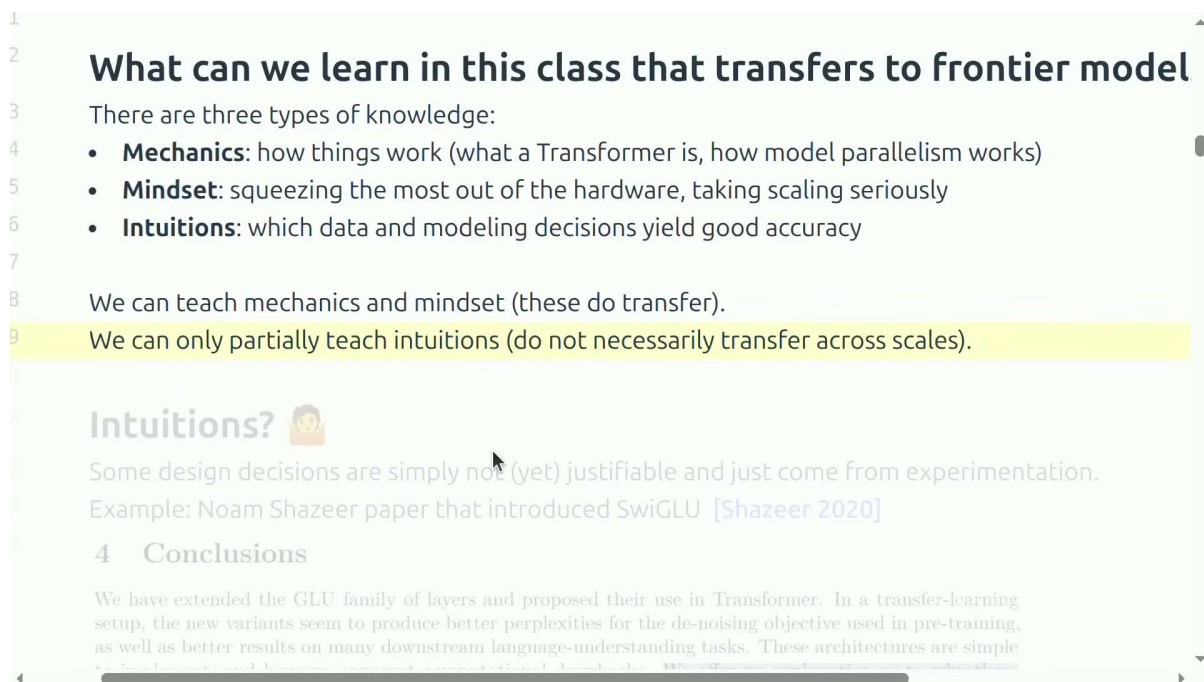


图 5：可迁移知识的三层结构：机制和效率思维较稳健，建模直觉需要谨慎外推。⁵

1.3 本章小结

“从零构建”是一种研究方法：通过实现建立跨层因果模型。小规模课堂无法复刻前沿训练，却能训练机制理解和效率思维；任何从小模型得到的质量直觉都应标注适用边界。

2 效率主线与“苦涩教训”

“苦涩教训”常被误读为“规模决定一切，算法不重要”。讲者给出的正确解释是：真正长期有效的是能够随资源扩展的算法。硬件、数据和算力增长不会替代算法改进；相反，规模越大，浪费一个百分点的成本越高。

$$\text{Accuracy} \approx \text{Efficiency} \times \text{Resources}.$$

- Accuracy：给定任务分布上的可用质量；
- Efficiency：单位数据、计算、显存和通信所换取的质量；
- Resources：可用数据、硬件、时间与资金。

⁵视频画面时间区间：00:08:00–00:08:15。

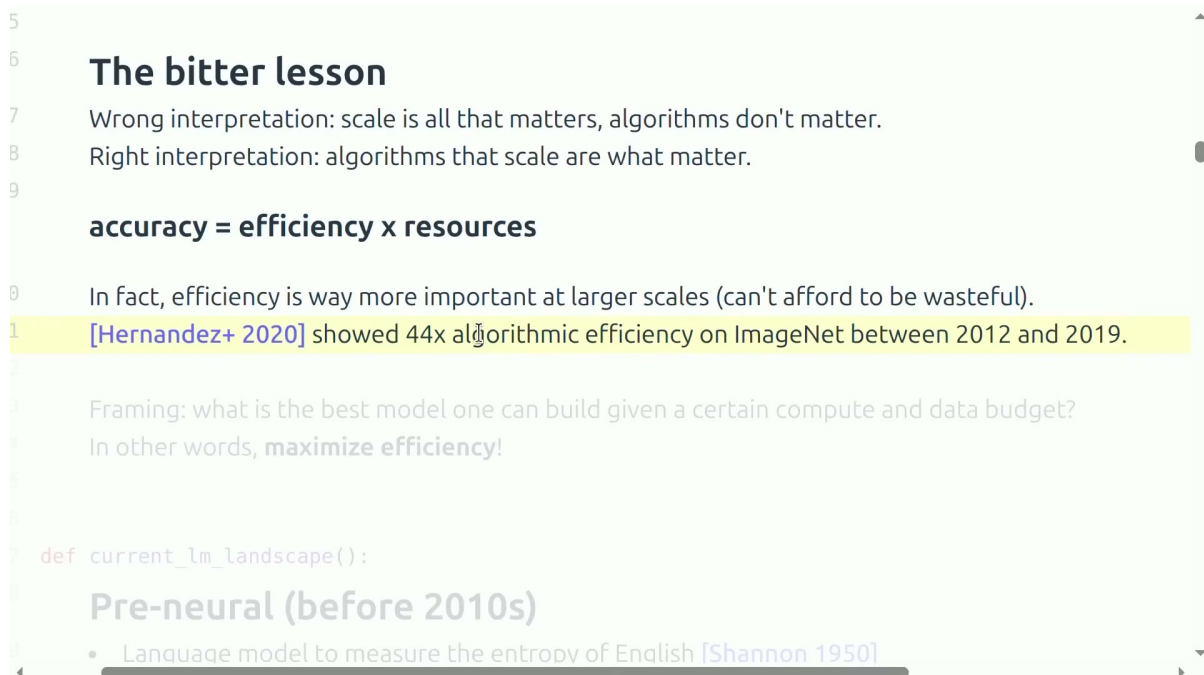


图 6: “苦涩教训”的课程版解释: 质量来自资源与算法效率的乘积, 而非单独依赖规模。⁶

2.1 同一套效率问题贯穿全栈

Tokenizer 用更有信息量的块缩短序列; 架构用 GQA、局部注意力或 MoE 调整计算与容量; 系统用 kernel fusion 和并行降低数据搬运; 数据过滤避免把昂贵更新浪费在低质量样本; 缩放定律用小实验选择大训练配方。它们看似属于不同章节, 本质都是同一个问题: **在固定资源预算下训练出最好的模型。**

效率不等于少做 FLOPs

真实效率同时取决于计算、显存容量、显存带宽、互连通信、并行度和可用内核。减少 FLOPs 若增加不规则访存或同步, 墙钟时间可能反而变差。课程后续会不断把理论计算量与硬件利用率分开核算。

2.2 本章小结

规模化并没有让算法失去意义, 而是放大了算法效率的价值。理解 CS336 最好的方式, 是把每一个组件都翻译成资源分配问题, 再用质量和墙钟成本共同验收。

3 语言模型发展史: 接口在变, 基本机制仍在

3.1 从统计模型到神经基础模型

早期语言模型用 n-gram 估计局部条件概率。2010 年代逐步形成今天的神经组件: LSTM 改善长期依赖, 神经语言模型学习分布式表示, seq2seq 和 attention 服务机器翻译, Adam 改变优化实

⁶视频画面时间区间: 00:10:45–00:11:00。

践，Transformer 用自注意力统一序列建模，MoE 与模型并行打开规模化通路。

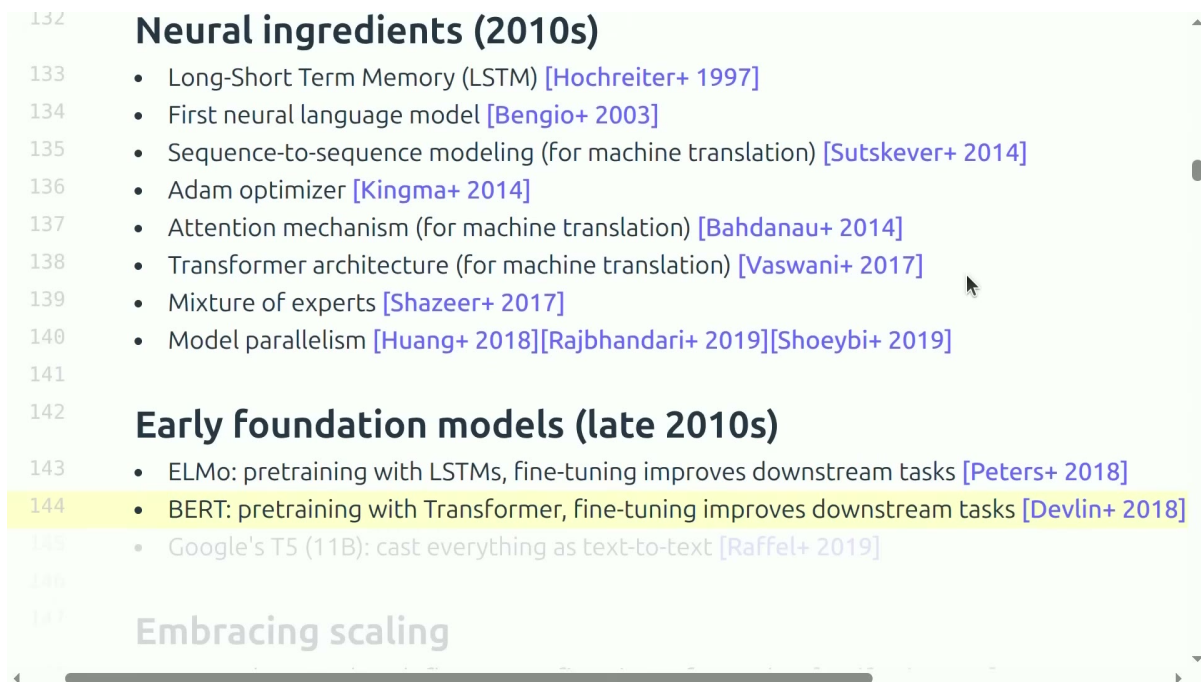


图 7：神经语言模型关键技术时间线：LSTM、神经 LM、seq2seq、Adam、attention、Transformer、MoE 与并行。⁷

随后，ELMo 与 BERT 建立“预训练后微调”的范式；GPT-2 暴露零样本能力；缩放定律给大训练带来可预测性；GPT-3 展示上下文学习；PaLM 和 Chinchilla 则推动计算最优训练讨论。这里的重点不是背诵模型名，而是理解接口与资源约束如何共同演化。

3.2 开放生态为何重要

开放权重只允许部署和评测；开放论文增加设计说明；开放代码允许复现实作；开放数据和开发过程才使因果追踪更完整。课程材料大量依赖 LLaMA、Mistral、DeepSeek、Qwen、Kimi、GLM 等开放权重模型，以及 OLMo、Nemotron、Marin 等更完整的开放项目。

⁷ 视频画面时间区间：00:13:30–00:13:45。

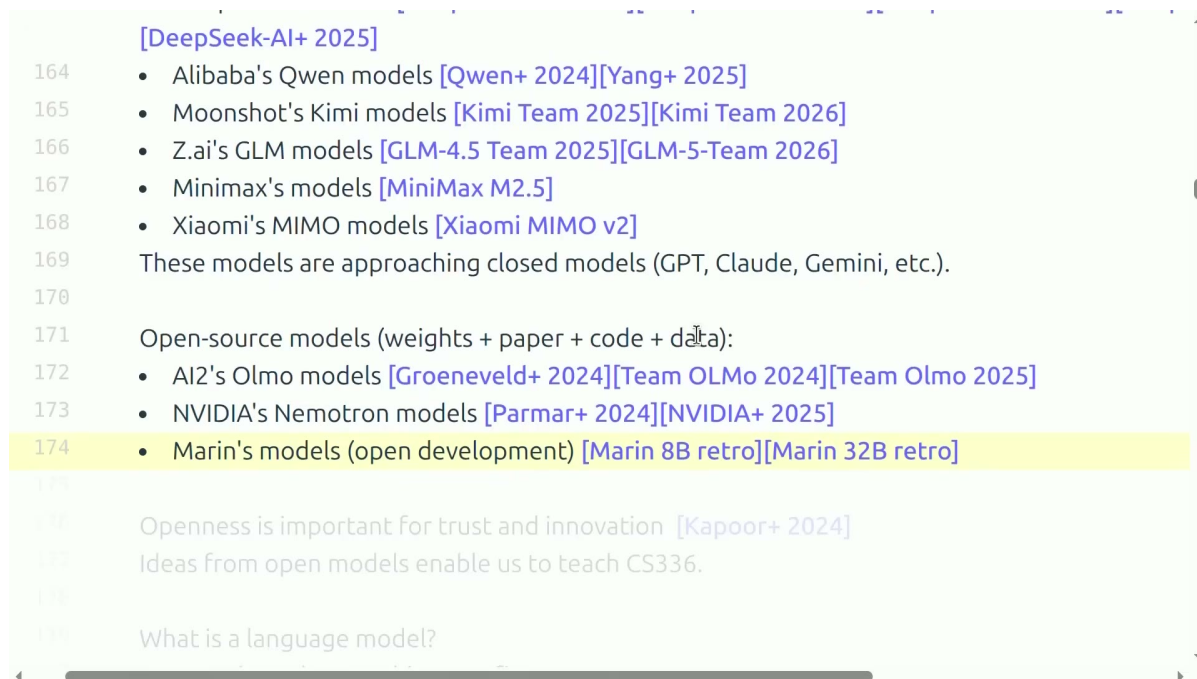


图 8: 开放模型版图: 开放权重、论文、代码和数据的程度不同, 能支持的研究问题也不同。⁸

“开放权重” 不等于 “开源训练”

拿到参数并不能回答数据如何清洗、失败实验如何处理、超参数为何选择、训练中断如何恢复。评估复现性时, 应明确区分权重、架构、训练代码、数据和开发日志五个层次。

3.3 语言模型接口的四次变化

2018 年, 语言模型是 “下载后微调的表示”; 2020 年, 它成为 “通过 prompt 调用的生成器”; 2022 年, 它成为 “可以对话的系统”; 2026 年, 智能体把模型变成能自主采取行动、调用工具和维护长轨迹的组件。接口变化很大, 但 attention、kernel、优化等基础仍然存在; 变化最快的是上下文长度和推理效率要求。

⁸视频画面时间区间: 00:16:45–00:17:00。

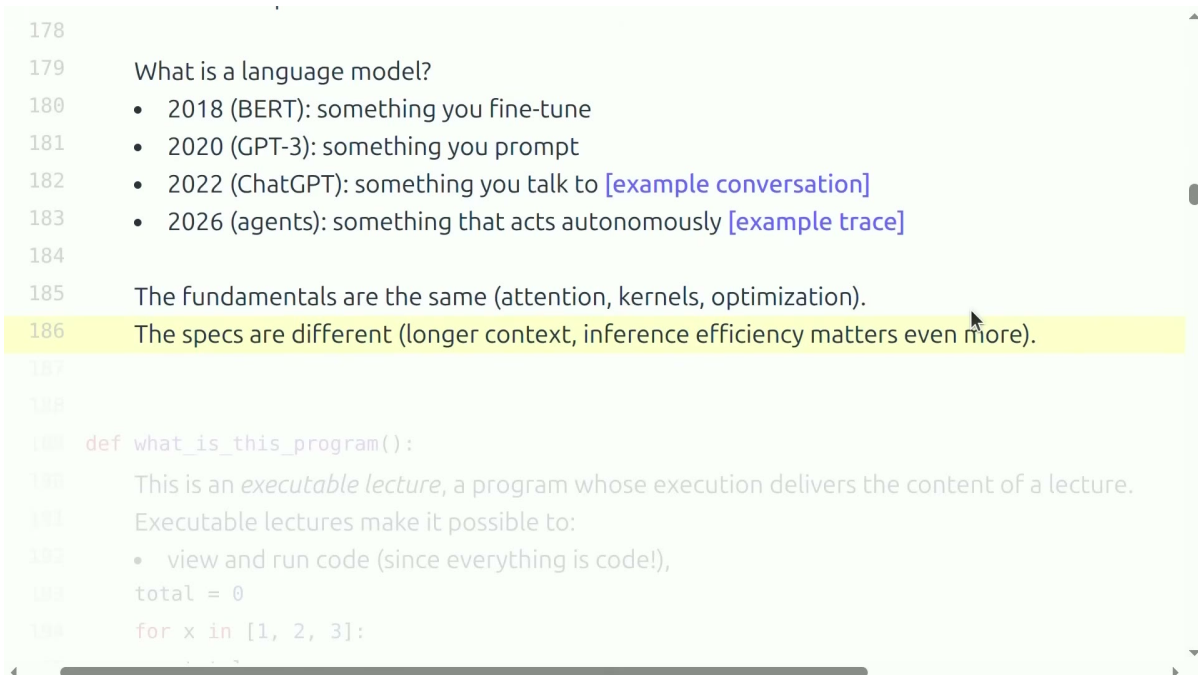


图 9: 语言模型接口从 fine-tune、prompt、chat 演化到 agent；底层机制稳定，系统规格持续变化。⁹

3.4 本章小结

语言模型的表面接口从概率估计一路演化到智能体，但表示、注意力、优化和系统执行仍是共同底座。开放生态让这些底座可被教学、验证和改进，也是从零构建课程能够成立的前提。

4 课程地图一：从 Tokenizer 到训练循环

CS336 用五个作业串起完整管线。Assignment 1 要求实现 BPE tokenizer、Transformer、交叉熵、AdamW 和训练循环，并在固定 B200 时间预算下优化困惑度。它不是把现成库拼起来，而是要求学生同时面对**表达力**、**稳定性**与**效率**三者的平衡。

4.1 Tokenization: 模型究竟在哪些“原子”上计算

原始输入是字节，模型处理的是整数 token 序列。Tokenizer 决定一个字符、词根、空格或代码片段占多少计算位置。BPE 把高频相邻字节组合成更大的块，常把约 1000 个字节压成约 250 个 token；同时，它把更多建模容量留给罕见或结构复杂的片段。

⁹视频画面时间区间：00:19:00–00:19:15。

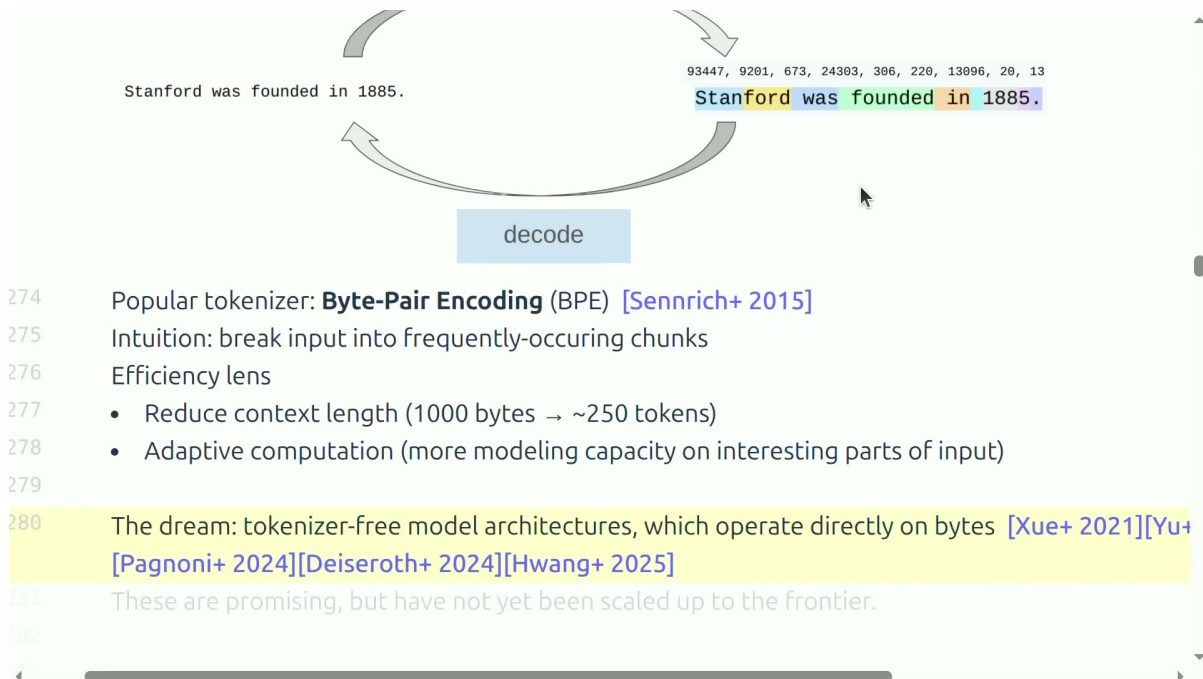


图 10: 课程中的 Tokenization: 在可逆编码前提下缩短上下文, 并进行数据驱动的自适应分块。¹⁰

4.2 模型架构: 从原始 Transformer 到现代组件

课程从原始 Transformer 出发, 再讨论 SwiGLU、RoPE、RMSNorm、QK-Norm、Pre-Norm、稀疏 / 局部注意力、GQA、MLA、状态空间模型和 MoE。架构问题不是组件越新越好, 而是每个选择如何改变计算、内存、稳定性、上下文传播和推理缓存。

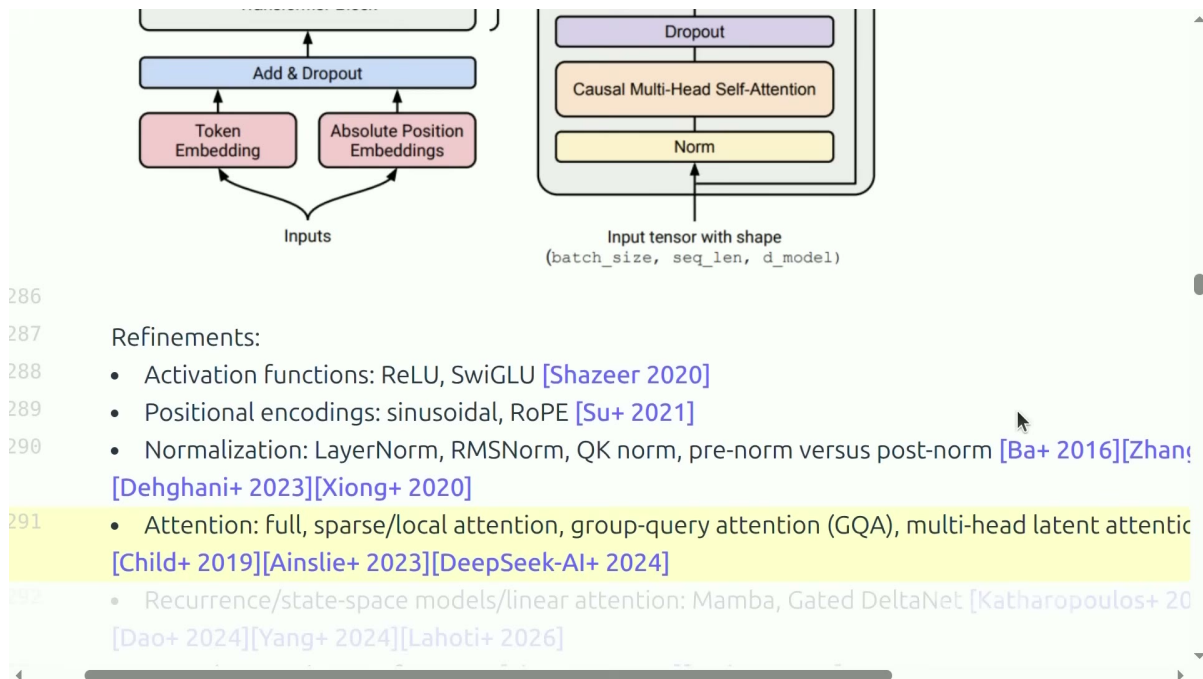


图 11: 现代语言模型架构的主要改进轴: 激活、位置、归一化、注意力、递归 / 状态空间与 MoE。¹¹

¹⁰ 视频画面时间区间: 00:29:15–00:29:30。

4.3 训练配方：参数如何被可靠地设置

训练配方包含损失函数、优化器、初始化、学习率日程、正则化、batch size 和 MoE 负载均衡。它们不能孤立调节：初始化改变激活尺度，学习率改变权重衰减的累计作用，batch size 改变梯度噪声，MoE 路由又改变专家负载。

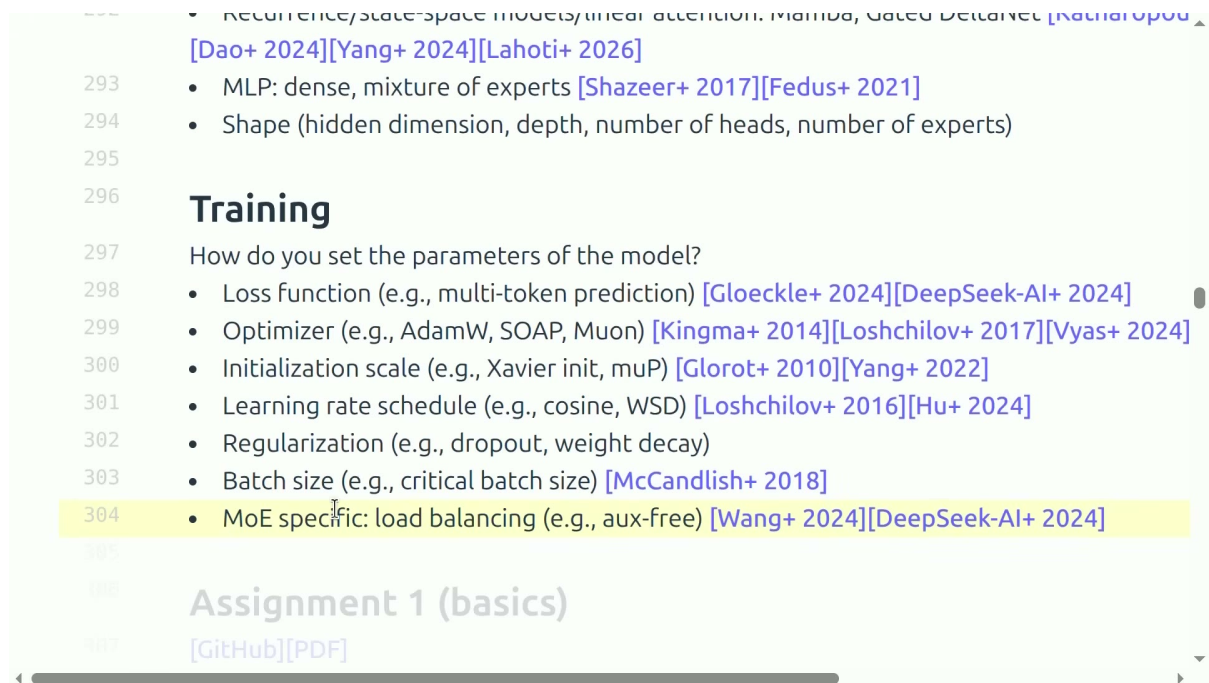


图 12：训练配方的耦合组件：损失、优化器、初始化、学习率、正则化、批大小与专家负载。¹²

Assignment 1 的验收不只看正确性

Tokenizer 要可逆且高效；Transformer 要通过形状和梯度测试；优化器要匹配公式；训练循环还要在固定时间预算下获得低困惑度。正确实现只是起点，最终要建立“质量—速度—资源”联合指标。

4.4 本章小结

基础作业已经包含一个完整语言模型系统：Tokenizer 决定计算单位，架构决定容量与信息流，训练配方决定参数轨迹。三者共同作用，不能把最终质量归因于单个组件。

5 课程地图二：系统、并行与推理

Assignment 2 把注意力从数学对象变成硬件程序：学生要实现 Triton kernel、分布式数据并行、优化器状态分片，并系统地 benchmark 和 profile。核心原则是减少数据搬运，而不只是减少算术操作。

¹¹ 视频画面时间区间：00:30:45–00:31:00。

¹² 视频画面时间区间：00:33:15–00:33:30。

5.1 资源核算与 Roofline 视角

模型参数必须从 HBM 搬到计算单元。以 B200 为例，bf16 峰值计算吞吐约为 PFLOP/s 量级，而显存带宽为 TB/s 量级；一个算子能否“喂饱”计算单元，取决于每搬运一个字节能做多少 FLOPs。

训练稠密 Transformer 的粗略算力可写为：

$$C \approx 6ND.$$

- C ：一次完整训练的浮点运算量；
- N ：模型参数量；
- D ：训练 token 数；
- 系数 6：前向、反向和参数梯度的常用近似合计。

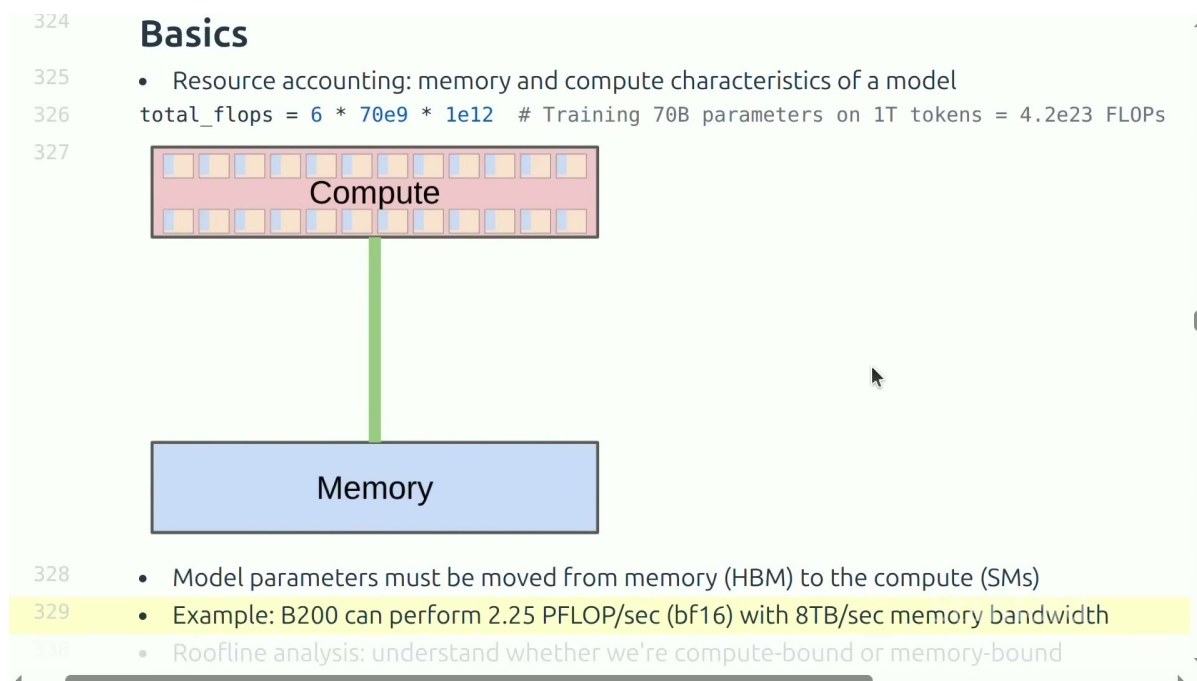


图 13: 资源核算的基本图景：计算单元与 HBM 之间的数据通路可能比理论 FLOPs 更早成为瓶颈。¹³

5.2 Kernel：把数学表达式组织成硬件友好程序

PyTorch 中每个 primitive 通常触发标准 kernel；自定义 kernel 可以融合中间步骤、复用片上存储、减少 HBM 往返。优化 FlashAttention 的本质便是改变循环与分块顺序，使注意力在不物化巨大中间矩阵的情况下完成相同计算。

¹³ 视频画面时间区间：00:37:30–00:37:45。

- Can write custom kernels to make GPUs go brrr
- Principle: organize computation to minimize data movement
- Naive: read HBM; compute A; write HBM; read HBM; compute B; write HBM
- Fused: read HBM; compute A and B; write HBM
- Strategies: operator fusion (matmul + activation), tiling (FlashAttention)
- Warp divergence, memory coalescing, bank conflicts, occupancy, bulk-async memory transfers
- Write kernels in CUDA/**Triton**/CUTLASS/ThunderKittens

Parallelism

- What if we have 1024 GPUs?
- Data movement between GPUs is even slower, but same 'minimize data movement' principle holds
- Use classic collective operations (e.g., gather, reduce, all-reduce)
- Shard memory (parameters, activations, gradients, optimizer states) across GPUs
- How to split computation: {data,tensor,pipeline,sequence,expert} parallelism

Inference

图 14: Kernel 优化的共同原则：融合操作、复用片上数据、减少 HBM 与计算单元之间的往返。¹⁴

5.3 并行：通信是跨 GPU 的数据搬运

当模型扩展到多 GPU，数据并行、张量并行、流水线并行、序列并行和专家并行会切分不同维度。GPU 互连通常比片上计算慢，因此并行方案的优劣不能只看每卡 FLOPs；还要看通信体积、同步频率、负载均衡和流水线气泡。

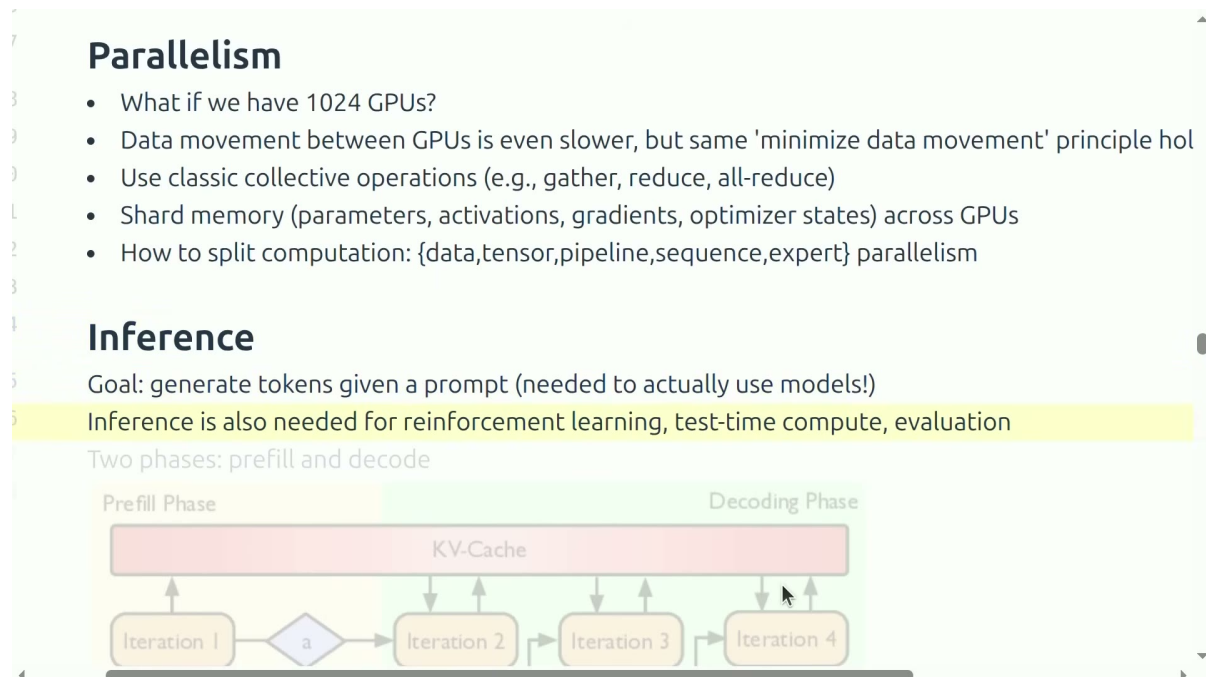


图 15: 多 GPU 并行仍遵循“最小化数据搬运”：共享参数、激活、梯度和优化器状态都带来通信。¹⁵

¹⁴ 视频画面时间区间：00:40:30–00:40:45。

5.4 推理：Prefill 与 Decode 是两种工作负载

Prefill 一次处理整段 prompt，矩阵规模大、并行度高，常偏计算受限；Decode 每次只生成一个 token，必须读取参数和 KV cache，且时间步之间串行，常偏显存带宽受限。量化、剪枝、蒸馏、投机解码、fused kernel 和连续批处理，分别从模型大小、候选生成和调度层面降低成本。

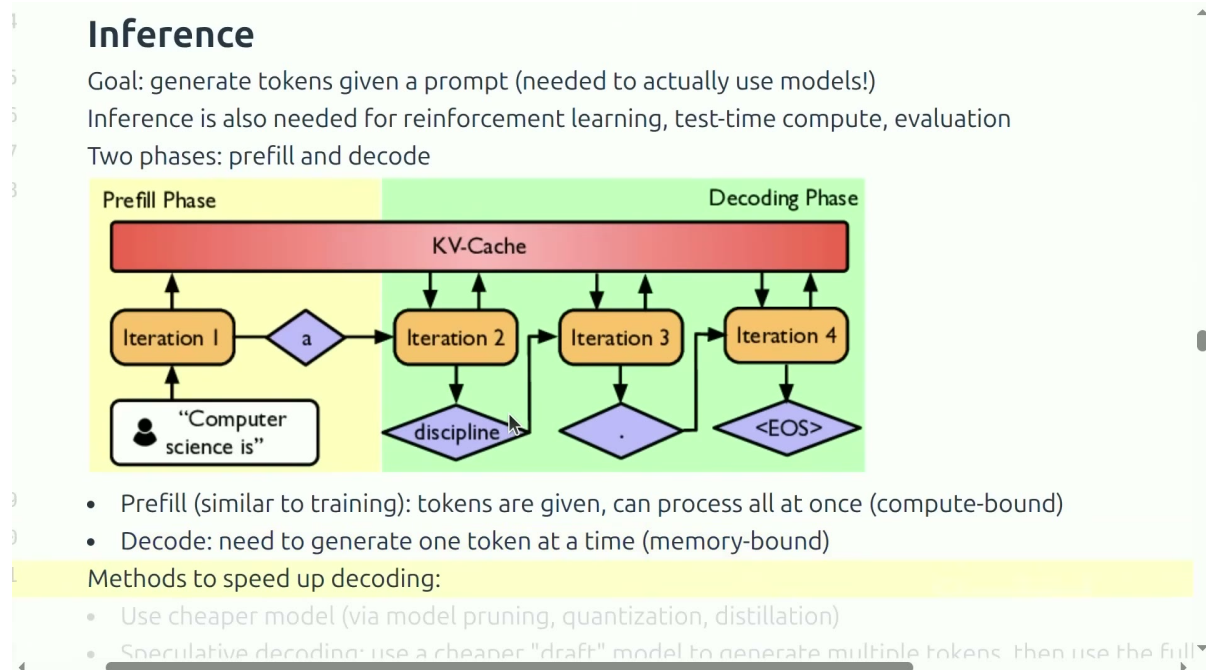


图 16: 自回归推理的两阶段：Prefill 可并行，Decode 逐 token 串行并逐步扩大 KV cache。¹⁶

训练吞吐不能代替推理指标

一个模型可以在训练时充分利用大矩阵，却在 Decode 时被 KV cache 和参数读取拖慢。部署评估至少要分别报告首 token 延迟、逐 token 延迟、并发容量和缓存字节 / token。

5.5 本章小结

系统章节把“效率”具体化为算术强度、数据搬运和通信。Kernel、并行与推理优化虽然处在不同层次，却共享一个问题：如何让昂贵数据在被搬运后得到更多有效复用。

6 课程地图三：缩放定律与可预测训练

前沿训练无法在目标规模上网格搜索超参数。缩放定律的关键转变，是从“某个固定规模的最优点”转向一条 **Scaling Recipe**：给定 FLOPs，自动产生模型宽深、batch、学习率、训练 token 等超参数。

¹⁵ 视频画面时间区间：00:41:45–00:42:00。

¹⁶ 视频画面时间区间：00:43:00–00:43:15。

6.1 用小实验优化大训练配方

流程包含三步：在多个较小算力预算上运行同一配方；拟合损失随算力的规律；把规律外推到目标规模。这样既能优化目标配方，也能在真正训练之前预注册预测损失。可预测性本身就是工程价值，因为它降低昂贵训练的决策风险。

```

3 def scaling_laws():
4     Setting: if you had 1e25 FLOPs of compute, what hyperparameters would you use to train a good m
5     Too expensive to do hyperparameter tuning at full scale!
6
7     Key conceptual shift: instead of a single scale, think of a scaling recipe (FLOPs → hyperparameters)
8     For a scaling recipe:
9     • Run experiments to compute the loss at various smaller scales (e.g., up to 1e24 FLOPs)
10    • Fit a scaling law to predict the loss of the scaling recipe at the target scale (e.g., 1e25 FLOPs)
11
12    Now you can:
13    1. Optimize the scaling recipe targeting a larger scale using smaller scale experiments
14    2. Predict the loss at the target scale before actually running the experiment!
15
16    Scaling laws don't happen automatically, they require careful construction of a scaling recipe.
17    Parameterize the model in a way to get hyperparameter transfer [Yang+ 2022]
18    Predictability is at least as important as optimality!
  
```

图 17: Scaling Recipe: 用多个小规模实验拟合规律，优化并预测目标规模的训练结果。¹⁷

6.2 等算力曲线：更大模型还是更多 token

固定 $C \approx 6ND$ 后，增大 N 就必须减少 D ，反之亦然。等算力实验在多个小预算下扫描模型大小，寻找每条曲线的最优点，再拟合最优 N 随 C 的变化。经典 Chinchilla 经验常写成 $D \approx 20N$ ，但它没有纳入推理成本：需要大量服务时，更小、训练更久的模型可能更经济。

¹⁷ 视频画面时间区间：00:47:15–00:47:30。

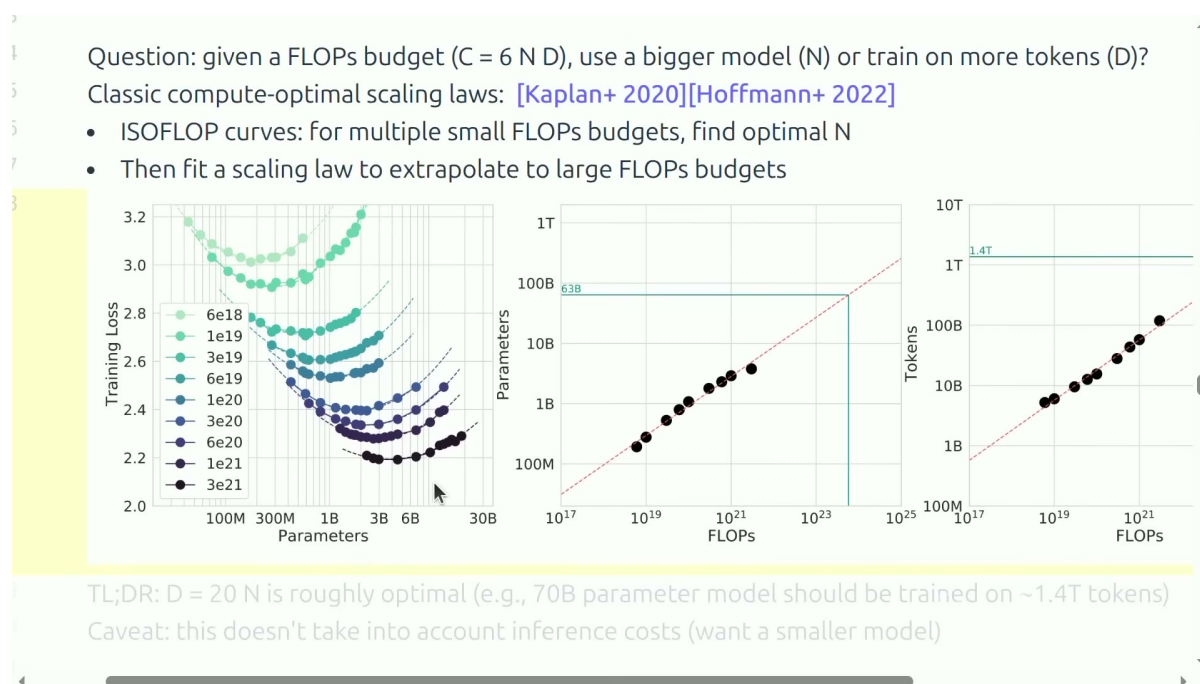


图 18: 等算力曲线: 在固定训练 FLOPs 下寻找模型参数与训练 token 的最佳分配。¹⁸

缩放规律不会自动出现

如果小实验改变了数据质量、参数化、优化稳定性或硬件利用率, 拟合到的曲线可能只是混合效应。可靠外推要求一条定义清楚、跨规模一致的 recipe, 并对超参数迁移做专门设计。

6.3 本章小结

缩放定律是决策工具, 不是自然常数。它通过小规模、受控、可比较的实验, 为大训练选择配方并量化风险; 推理成本和数据约束仍可能改变“计算最优”的答案。

7 课程地图四: 评测、数据与对齐

7.1 评测服务于两类不同目的

内部评测用于指导开发, 重视跨规模平滑和相对差异; 外部评测用于衡量真实用例, 重视生态有效性和绝对质量。困惑度适合连续监控, 但应尽量使用未被网络污染的私有文本; GPQA、HLE、SWE-Bench、Terminal-Bench 等高级任务则覆盖知识、推理和智能体能力。

7.2 数据不是“从天上掉下来”的

预训练数据来自网页、书籍、论文、代码等异构来源, 原始形态可能是 HTML、PDF 或目录而非干净文本。数据权利、许可和公平使用也是管线的一部分。多来源混合时, 体量最大的来源并不一定应该拥有最大的训练权重。

¹⁸ 视频画面时间区间: 00:50:00–00:50:15。

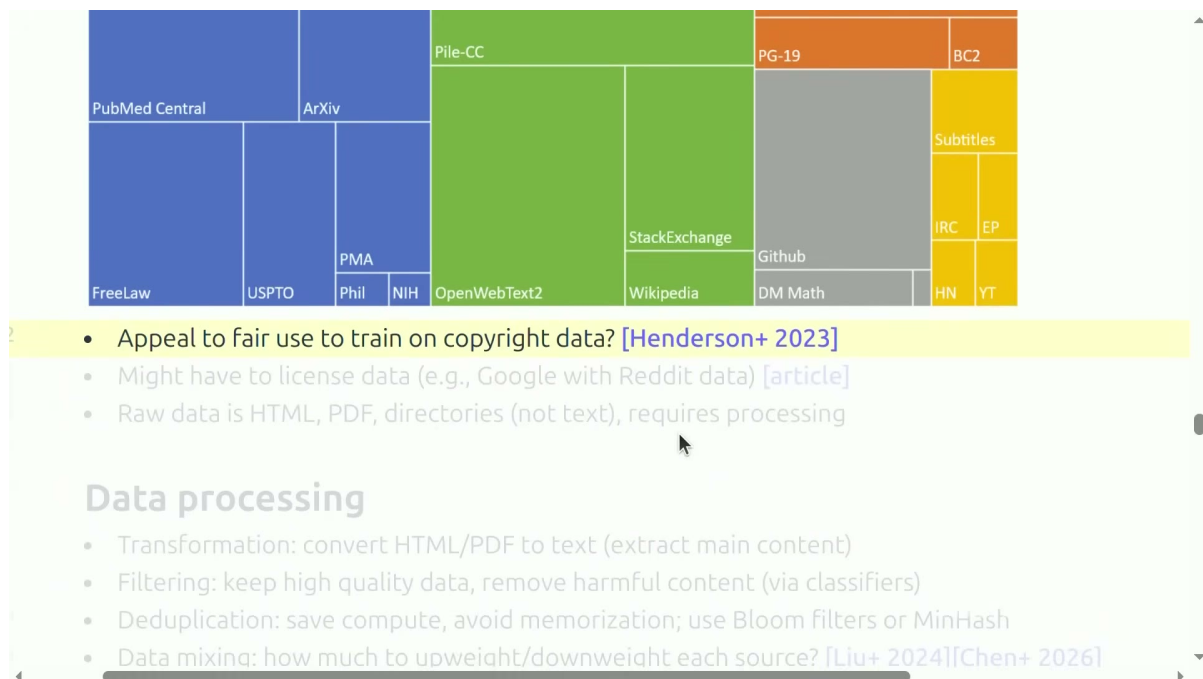


图 19: 典型预训练语料的来源构成：网页、论文、书籍、代码与其他领域数据共同组成混合分布。¹⁹

7.3 数据处理是一条有损决策链

转换把 HTML / PDF 变成正文；过滤移除低质量或有害内容；去重节省计算并降低记忆；混合调节不同来源权重；重写和合成使原始数据更接近目标任务。每一步都可能删除有价值的长尾，也可能把分类器偏见放大，因此必须用下游评测和数据审计闭环验证。

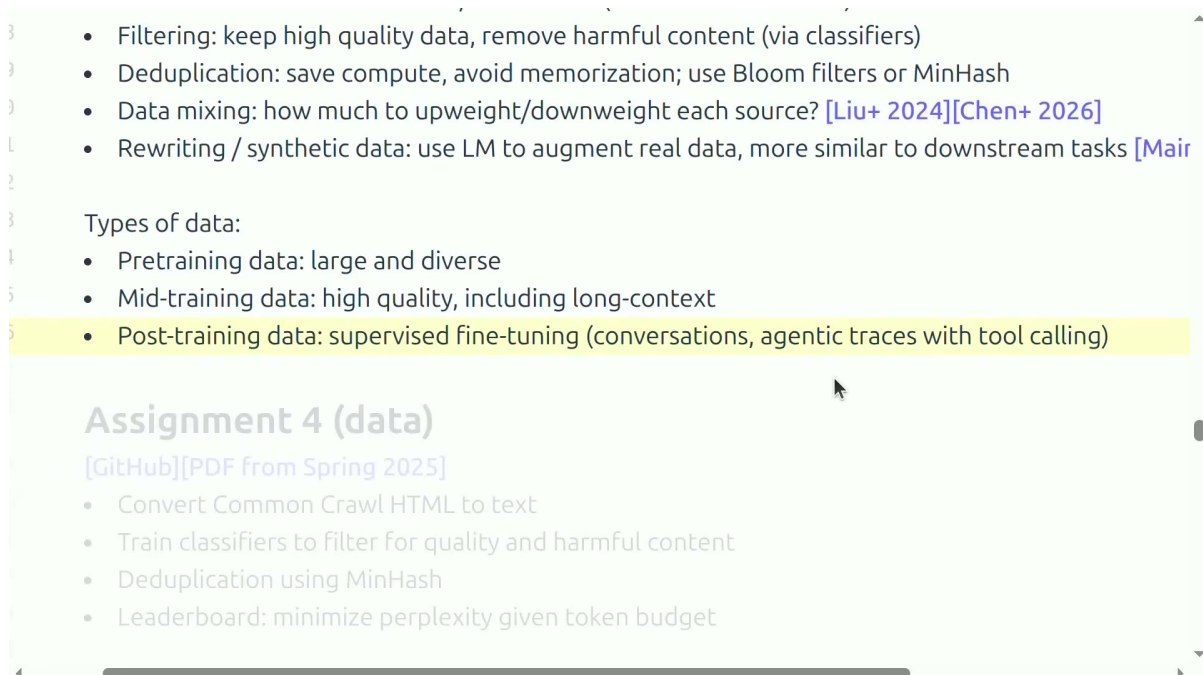


图 20: 数据处理主流程：转换、过滤、去重、混合与合成；不同阶段的数据承担不同训练目标。²⁰

¹⁹视频画面时间区间：00:57:30–00:57:45。

7.4 对齐：当评价比生成更容易

预训练使用完整监督——每个下一个 token 都有标签；对齐则利用弱监督：模型生成候选，由人类、验证器或 LM judge 打分，再更新模型偏好。DPO 直接使用偏好对，PPO 引入强化学习，GRPO 去掉价值函数。难点是奖励投机、训练不稳定，以及大规模 rollout 对推理系统的压力。

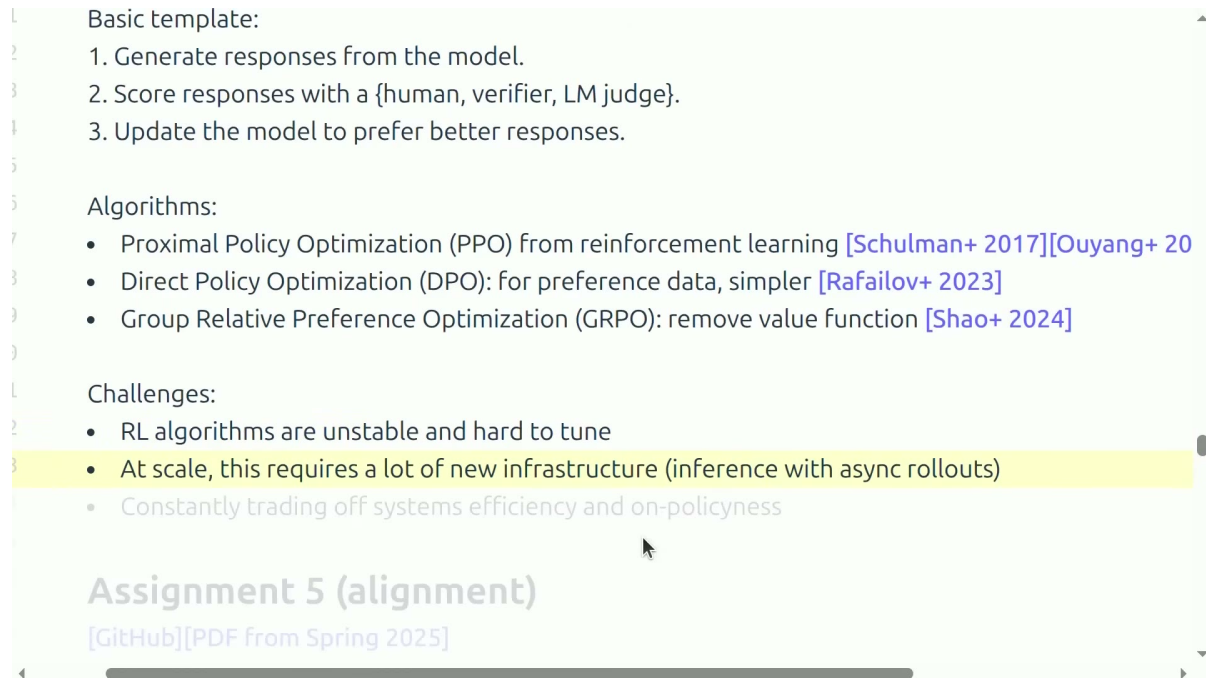


图 21：对齐基本循环与算法家族：生成、评分、更新，并在监督强度与在线性之间权衡。²¹

7.5 一条贯穿五个作业的主线

课程最后再次强调：当前训练往往受计算限制，因此 tokenizer、架构、数据过滤和缩放实验都围绕“不要浪费计算”展开；未来若高质量数据成为主要瓶颈，最优选择可能改变。资源约束不是背景条件，而是决定架构和算法价值的自变量。

²⁰ 视频画面时间区间：00:59:30–00:59:45。

²¹ 视频画面时间区间：01:01:45–01:02:00。

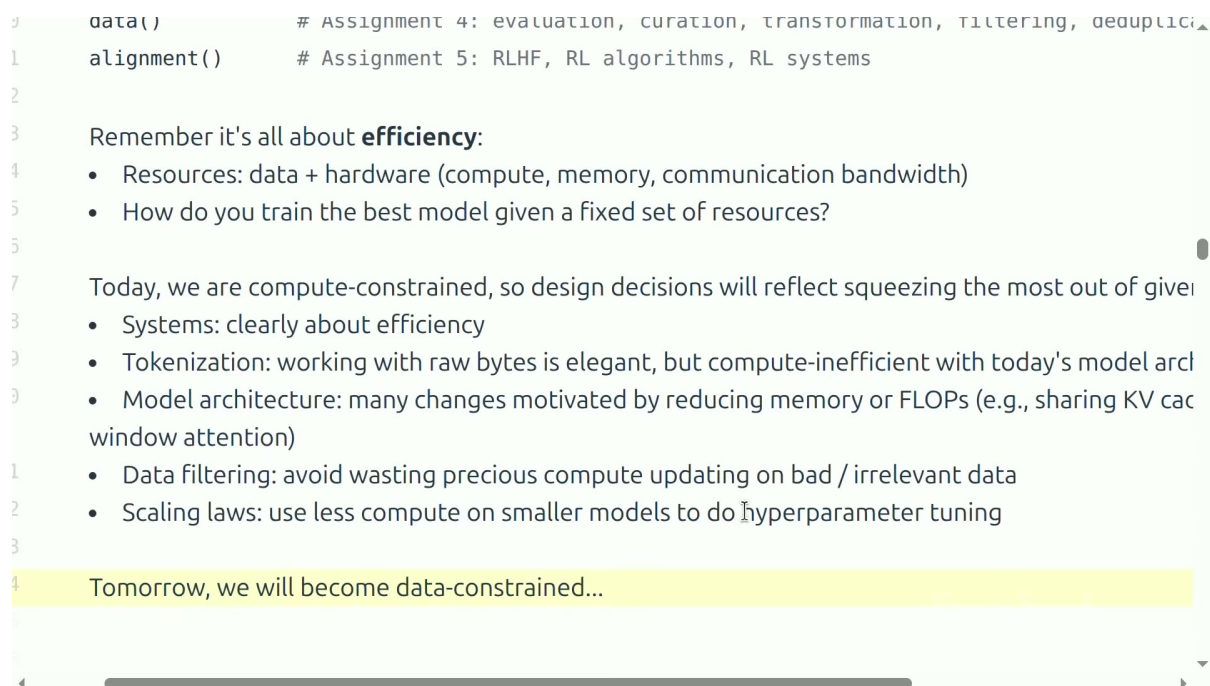


图 22: 全课程效率主线: 数据与硬件给出资源边界, 系统、Tokenizer、架构、数据过滤和缩放共同分配预算。²²

7.6 本章小结

评测定义目标, 数据决定学习分布, 对齐利用弱监督修正行为。三者必须闭环: 没有可靠评测就无法判断数据处理与对齐是否真的改善模型; 没有资源核算, 改进也可能无法扩展。

8 Tokenization 基础: 可逆表示与计算预算

语言模型对整数序列定义概率分布, 而用户输入通常是 Unicode 字符串。Tokenizer 必须在两种表示之间建立可逆映射:

$$E: \mathcal{S} \rightarrow \mathbb{N}^*, \quad D: \mathbb{N}^* \rightarrow \mathcal{S}, \quad D(E(s)) = s.$$

- $\mathcal{S}$: 合法输入字符串集合;
- E : encode, 把字符串映射为有限整数序列;
- D : decode, 把整数序列还原为字符串;
- $D(E(s)) = s$: round-trip 条件, 任何合法文本都不能在编码后丢失信息。

²² 视频画面时间区间: 01:04:30–01:04:45。

Listing 1: Tokenizer 的最小接口：编码与解码必须互为逆过程

```

1 class Tokenizer:
2     def encode(self, text: str) -> list[int]:
3         raise NotImplementedError
4
5     def decode(self, ids: list[int]) -> str:
6         raise NotImplementedError

```

8.1 现实 Tokenizer 的几个反直觉现象

常见 BPE Tokenizer 会把前导空格和单词合成一个 token，因此句首的“hello”与句中的“hello”可能不同；数字常按固定长度片段拆分；中英文和 emoji 的 token 数差异也很大。这些不是语言学真理，而是训练语料、预切分正则表达式和合并表共同产生的工程结果。

• A word and its preceding space are part of the same token (e.g., " world").
 • A word at the beginning and in the middle are represented differently (e.g., "hello hello").
 • Numbers are tokenized into a fixed number of digits.

Here's the GPT-5 tokenizer from OpenAI (tiktoken) in action:

```

tokenizer = get_gpt5_tokenizer()
string = "Hello, 🌍! 你好!"

```

string	indices
"Hello, 🌍! 你好!"	[13225, 11, 130321, 235, 0, 220, 177519, 0]
reconstructed_string	"Hello, 🌍! 你好!"

Check that encode() and decode() roundtrip:

```

indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
assert string == reconstructed_string

```

Compression ratio: number of bytes per token

```

compression_ratio = get_compression_ratio(string, indices)

```

The larger the compression ratio, the shorter the sequence (good since attention is quadratic in sequence length).

图 23: GPT 系 Tokenizer 的 round-trip 示例：Unicode 字符串被映射为整数序列后可无损恢复。²³

8.2 压缩率与词表大小

讲者把压缩率定义为每个 token 覆盖的 UTF-8 字节数：

$$r = \frac{|s_{\text{UTF8}}|}{|E(s)|}.$$

- r ：平均每个 token 表示的字节数，越大表示序列越短；
- $|s_{\text{UTF8}}|$ ：字符串编码后的字节数；

²³ 视频画面时间区间：01:07:15–01:07:30。

- $|E(s)|$: Tokenizer 输出的 token 数。

短序列能降低 attention 的二次成本,但提升压缩率通常需要更大词表;大词表会扩大 embedding 和输出 softmax, 且让低频 token 更稀疏。Tokenizer 因此是在**序列长度、词表参数和统计复用**之间分配预算。

The screenshot shows a Jupyter Notebook cell with the following code and output:

```

2 string = "Hello, 🌍! 你好!"
3
4 Check that encode() and decode() are round-trip:
5 indices = tokenizer.encode(string)
5 reconstructed_string = tokenizer.decode(indices)
7 assert string == reconstructed_string
3
3 Compression ratio: number of bytes per token
3 compression_ratio = get_compression_ratio(string, indices)
1 The larger the compression ratio, the shorter the sequence (good since attention is quadratic in seq length).
2 One could increase compression ratio by increasing vocabulary size (number of possible token values increases), leading to sparsity.
vocabulary_size = tokenizer.n_vocab

Let's take a look at the actual vocabulary: [vocab]
output_tokenizer(tokenizer, "var/gpt5_tokenizer_vocab.txt")

```

The output of the code is displayed in a light blue box:

```

string = "Hello, 🌍! 你好!"
indices = [13225, 11, 130321, 235, 0, 220, 177519, 0]
reconstructed_string = "Hello, 🌍! 你好!"
compression_ratio = 2.5

```

图 24: 压缩率与词表大小的权衡: 更大词表可缩短序列, 却会扩大稀疏表示和输出层。²⁴

8.3 本章小结

Tokenizer 首先必须可逆, 其次才比较效率。压缩率高能缩短上下文, 但词表过大又会浪费参数和样本; 任何方案都应同时报告 round-trip、token 数、词表规模和下游成本。

9 三种直接方案为什么都不理想

9.1 字符级: 语义直观, 但词表大且长尾严重

字符级 Tokenizer 把每个 Unicode code point 当作 token。实现简单、round-trip 清晰, 但 Unicode 字符约有十万量级, 许多字符极少出现; 模型要为罕见字符保留独立 embedding, 却很难获得足够训练信号。与此同时, 一个字符不一定对应一个字节, 跨语言压缩率差异明显。

²⁴ 视频画面时间区间: 01:08:00–01:08:15。

```

4  assert chr(97) == "a"
5  assert chr(127757) == "🌍"

5  Now let's build a Tokenizer and make sure it round-trips
3  tokenizer = CharacterTokenizer()
3  string = "Hello, 🌍! 你好!"
3  indices = tokenizer.encode(string) # call ord
1  reconstructed_string = tokenizer.decode(indices) # call chr
2  assert string == reconstructed_string

4  There are approximately 150K Unicode characters. [Wikipedia]
5  vocabulary_size = max(indices) + 1 # This is a lower bound
5  Problem 1: this is a very large vocabulary.
7  Problem 2: many characters are quite rare (e.g., 🌍), which is inefficient use of the vocabulary.
3  compression_ratio = get_compression_ratio(string, indices)
3  This tokenizer is the worst of both worlds (large vocabulary, low compression ratio).

```

string = "Hello, 🌍! 你好!"

indices = [72, 101, 108, 108, 111, 44, 32, 127757, 33, 32, 2032]

reconstructed_string = "Hello, 🌍! 你好!"

vocabulary_size = 127758

图 25: 字符级 Tokenizer: 直接使用 Unicode code point, 词表很大且大量字符低频。²⁵

9.2 字节级: 词表固定, 却把序列拉得过长

UTF-8 把任何字符串编码为 0–255 的字节序列, 因此字节级方案只需 256 个 token, 不存在未知字符。然而每个 token 只承载一个字节, 压缩率恒为 1; 中文和 emoji 需要多个 token, 序列很长, 而标准 attention 成本随长度平方增长。

```

3  string = "Hello, 🌍! 你好!"
4  indices = tokenizer.encode(string)
5  reconstructed_string = tokenizer.decode(indices)
5  assert string == reconstructed_string
7  vocabulary_size = 256
3  compression_ratio = 1
3  The compression ratio is terrible, which means the sequences will be too long.
3  Given that the context length of a Transformer is limited (since attention is quadratic), this is not lo

1  def word_tokenizer():
3  Another approach (closer to what was done classically in NLP) is to split strings into words.
3  string = "I'll say supercalifragilisticexpialidocious!"

```

string = "Hello, 🌍! 你好!"

indices = [72, 101, 108, 108, 111, 44, 32, 240, 159, 140, 141, 33, 32, 228, 189, 160, 229]

reconstructed_string = "Hello, 🌍! 你好!"

vocabulary_size = 256

compression_ratio = 1

图 26: 字节级 Tokenizer: 词表仅 256, 但压缩率为 1, 长序列会放大 attention 成本。²⁶

²⁵ 视频画面时间区间: 01:09:15–01:09:30。

9.3 词级：压缩好，却无法封闭词表

按正则表达式切词能得到人类可解释的单元，压缩率也较高；但专有名词、拼写变化、复合词、代码标识符和新词不断增长，词表没有自然上限。测试时未见词只能映射成 UNK，这会丢失原始信息并污染困惑度计算。

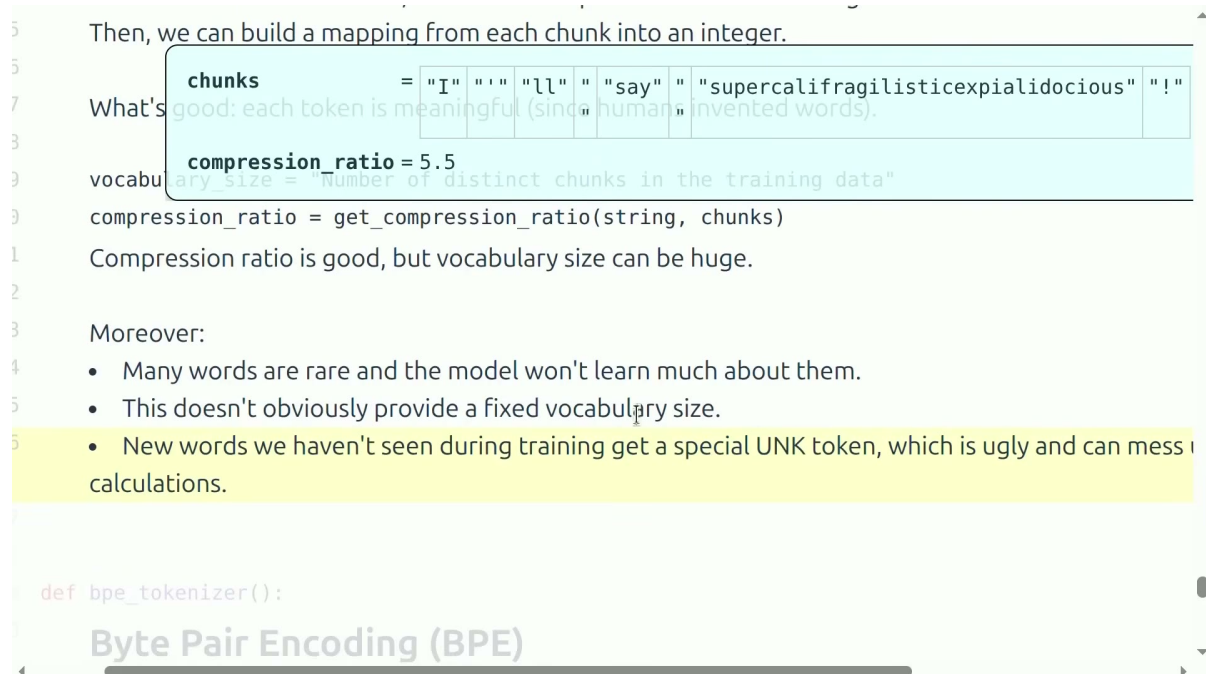


图 27：词级 Tokenizer：语义单元清晰，但长尾词和未登录词使固定词表难以成立。²⁷

三种方案的共同矛盾

字符级浪费词表，字节级浪费序列位置，词级无法覆盖长尾。理想方案要以小而封闭的基础字节集保证可逆，同时让高频片段共享更大的 token 以提高压缩率。

9.4 本章小结

字符、字节和词分别优化了可解释性、封闭词表和压缩率中的一部分，却无法同时满足三者。BPE 的价值正是用数据驱动的多尺度单元连接这三个极端。

10 Byte Pair Encoding：从字节学习可变粒度词表

10.1 基本直觉

BPE 最初是压缩算法，后来用于神经机器翻译并被 GPT-2 普及。训练从 256 个字节 token 开始，反复找出当前序列中最常见的相邻 token 对，把它合并为新 token。高频序列很快变成单个 token，罕见序列仍可退化为原始字节，所以不存在 UNK。

²⁶ 视频画面时间区间：01:10:30–01:10:45。

²⁷ 视频画面时间区间：01:11:45–01:12:00。


```
def bpe_tokenizer():
```

Byte Pair Encoding (BPE)

The BPE algorithm was introduced by Philip Gage in 1994 for data compression. [\[article\]](#)

It was adapted to NLP for neural machine translation. [\[Sennrich+ 2015\]](#)

(Previously, papers had been using word-based tokenization.)

BPE was then used by GPT-2. [\[Radford+ 2019\]](#)

Basic idea: *train* the tokenizer on raw text to construct a vocabulary tailored to the data.

Intuition: common sequences of bytes are represented by a single token, rare sequences are represented by many tokens.

Sketch: start with each byte as a token, and successively merge the most common pair of adjacent tokens.

Training the tokenizer

```
string = "the cat in the hat"
```

```
params = train_bpe(string, num_merges=3)
```

图 28: BPE 的核心: 从字节出发, 按语料频率逐步构建可变粒度词表。²⁸

设当前 token 序列为 z , 相邻对计数为 $c_z(a, b)$, 第 t 次合并选择:

$$(a_t, b_t) = \arg \max_{(a, b)} c_z(a, b), \quad v_t = 256 + t.$$

- $c_z(a, b)$: token 对 (a, b) 在当前序列中的相邻出现次数;
- (a_t, b_t) : 第 t 轮最常见的相邻对;
- v_t : 为该合并片段分配的新 token ID;
- 每轮替换后要重新统计, 因为合并会改变邻接关系。

²⁸视频画面时间区间: 01:12:45–01:13:00。

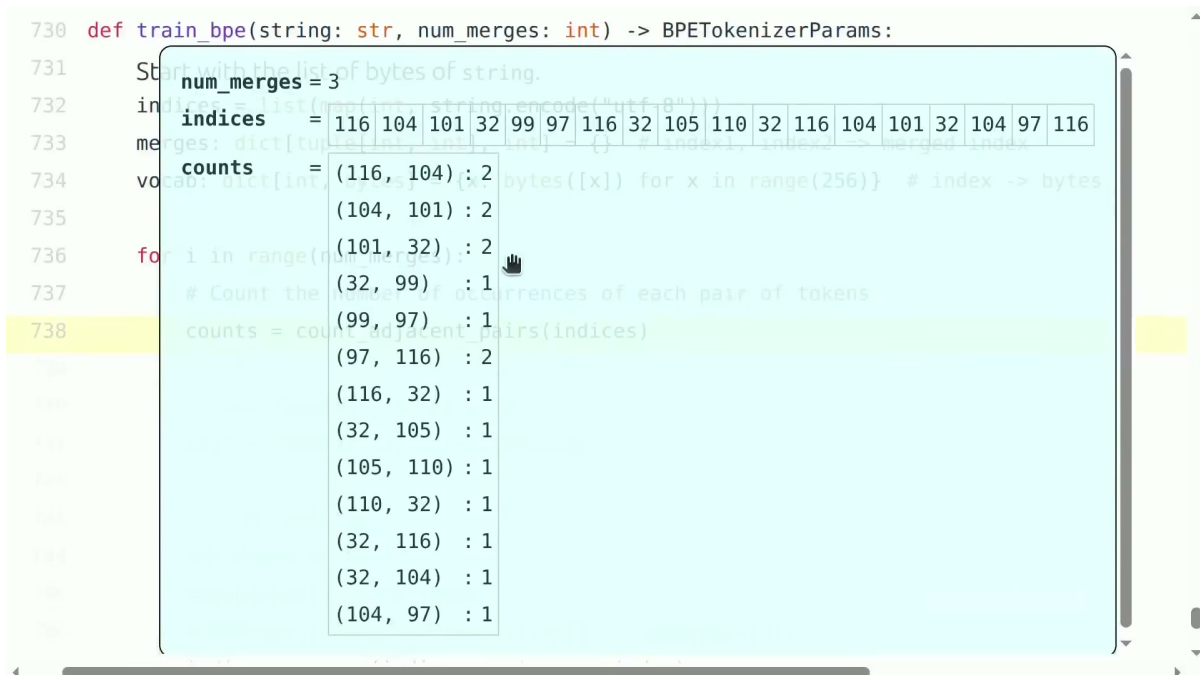


图 29: BPE 训练过程：统计相邻对、选择最高频组合、创建新 ID 并替换全部匹配。²⁹

10.2 最小实现与复杂度

Listing 2: 教学版 BPE 训练循环，生产实现需使用增量计数和高效索引

```

1 tokens = list(text.encode("utf-8"))
2 merges = {}
3 vocab = {i: bytes([i]) for i in range(256)}
4
5 for step in range(num_merges):
6     counts = count_adjacent_pairs(tokens)
7     pair = max(counts, key=counts.get)
8     new_id = 256 + step
9     merges[pair] = new_id
10    vocab[new_id] = vocab[pair[0]] + vocab[pair[1]]
11    tokens = merge_all(tokens, pair, new_id)

```

朴素实现每轮扫描全序列并重算全部 pair，训练和编码都很慢。Assignment 1 要求只更新受合并影响的计数、只检查相关 merge，并正确保护特殊 token。真正困难的地方通常不是十行算法，而是大语料上的数据结构、并行 I/O、预切分和确定性。

10.3 用合并表编码新文本

训练结果由基础词表和有序 merge 规则组成。编码新字符串时先转为字节，再按训练顺序应用相关合并；解码则把每个 token ID 映射回字节串并拼接，最后用 UTF-8 还原字符串。

²⁹ 视频画面时间区间：01:13:30–01:13:45。

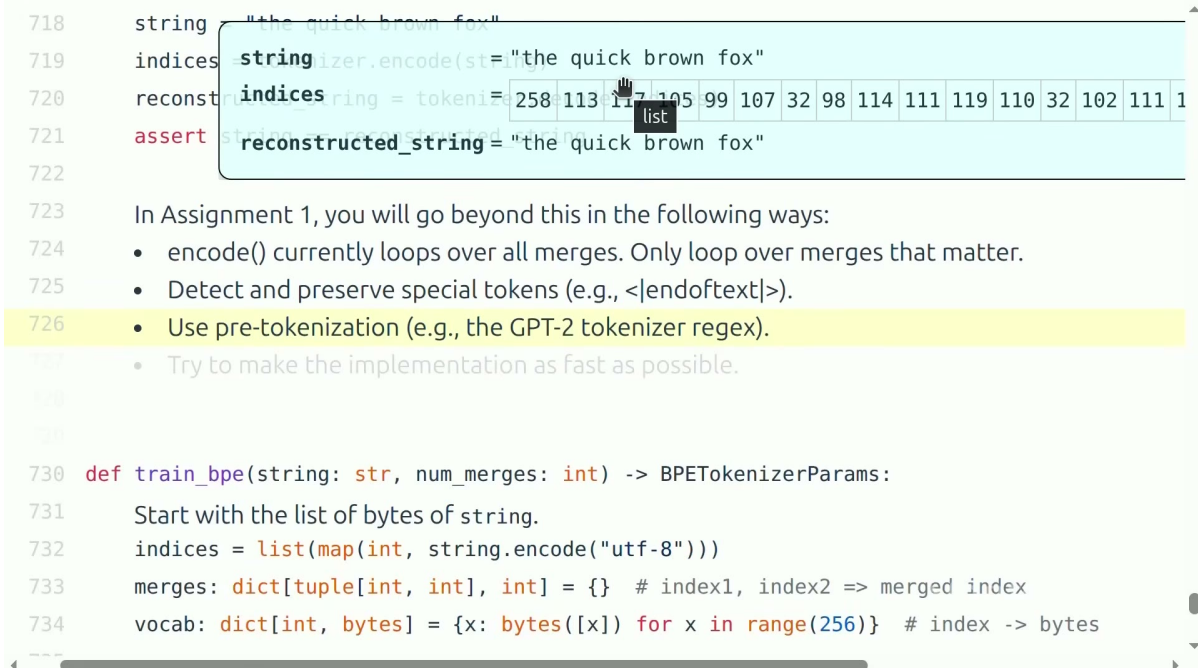


图 30: 使用训练后的 BPE: 对新文本应用合并规则, 编码结果仍能通过字节词表无损解码。³⁰

Tokenizer 是模型协议的一部分

训练、推理和评测必须使用完全相同的词表、merge 顺序、预切分规则与特殊 token 定义。只替换 Tokenizer 而不重训 embedding, 会让所有 token ID 的语义失效; 评测时忽略不同 Tokenizer 的 token 数, 也会误比较困惑度和成本。

10.4 Tokenizer-free 的长期方向

BPE 仍是独立于模型的预处理启发式。更理想的系统可能直接从字节学习分块, 使块边界与模型目标共同训练; 但现有 Transformer 对长字节序列的成本太高, 这类方案尚未在前沿规模普遍验证。无论未来采用何种机制, 都要满足两个条件: 模型在有意义的块上计算, 且块大小能随信息密度变化。

³⁰ 视频画面时间区间: 01:17:00–01:17:15。

```

tokenizer()

try:
    tokenizer: strings ↔ tokens (indices)
    character-based, byte-based, word-based tokenization are highly suboptimal
    BPE is an effective heuristic that is data-driven
    tokenization is a separate step, maybe one day do it end-to-end from bytes...

    whatever solution needs to satisfy:
    model (e.g., Transformer) should operate on chunks (abstractions) of the sequence (text, video, DNA, etc.)
    chunk sizes should be variable (allocate more model capacity to interesting chunks)

    characterTokenizer(Tokenizer):
    """Represent a string as a sequence of Unicode code points."""
    @code(self, string: str) -> list[int]:
    return list(map(ord, string))

```

图 31: Tokenization 总结: BPE 是有效的数据驱动启发式, 未来方向是端到端学习可变粒度分块。³¹

10.5 本章小结

BPE 用字节保证可逆, 用频繁合并提高压缩率, 用固定 merge 数控制词表规模。它解决了字符、字节和词级方案的主要矛盾, 但训练效率、特殊 token、预切分和跨语言公平性仍需工程化处理。

11 总结与延伸

11.1 讲者总结

Lecture 1 建立了整门课的坐标系: 从零构建不是为了复刻闭源前沿模型, 而是为了掌握可迁移的机制和效率思维; 课程的每个模块都在固定资源下平衡表达力、稳定性与效率。分词作为第一个具体模块, 展示了这种方法: 把抽象的“文本输入”拆成可逆性、词表、序列长度和计算成本, 再用代码验证权衡。

11.2 结构化提炼

1. **抽象必须可下钻。**高层 API 提升生产率, 但研究创新需要能追到数据、数值和系统边界。
2. **效率是乘数。**规模越大, 算法、数据与硬件利用率的一点改进价值越高。
3. **预测性与最优性同样重要。**稳定的缩放 recipe 能在投入大训练前量化结果与风险。
4. **Tokenizer 是资源分配器。**它决定每种文本消耗多少序列位置, 也决定 embedding 和 softmax 的词表成本。

³¹ 视频画面时间区间: 01:18:30–01:18:45。

5. 实现暴露隐含假设。Round-trip、特殊 token、merge 顺序和数据结构都不是公式里自动包含的细节。

Lecture 1 的一句话结论

真正的 “from scratch” 不是拒绝现成工具，而是让每个抽象都能被解释、测量和替换；从 Tokenizer 开始，把语言模型重新看成一套受资源约束的可执行系统。

11.3 建议练习

1. 为同一份中英代码混合文本计算字符、字节和 BPE 的 token 数、压缩率与 round-trip。
2. 实现朴素 BPE 后，用 profile 找到每轮全量计数的瓶颈，再设计增量更新结构。
3. 固定模型和训练 token 数，只改变词表大小，比较序列长度、embedding 参数、softmax 时间与验证损失。
4. 选一个架构改动，分别写出它对 FLOPs、显存容量、数据搬运、训练稳定性和推理缓存的影响。

11.4 拓展阅读

- CS336 Spring 2026: <https://cs336.stanford.edu/>
- Lecture 1 可执行课件：
https://github.com/stanford-cs336/lectures/blob/main/lecture_01.py
- Assignment 1 (Basics): <https://github.com/stanford-cs336/assignment1-basics>
- Neural Machine Translation of Rare Words with Subword Units: <https://arxiv.org/abs/1508.07909>
- The Bitter Lesson: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>
- Let's build the GPT Tokenizer (Karpathy): <https://www.youtube.com/watch?v=zduSFxRajkE>

11.5 本章小结

课程全景和 Tokenization 共同展示了 CS336 的方法论：先明确资源与不变量，再实现、测量、比较。后续架构、系统、数据与对齐章节都将重复这套闭环，只是约束和瓶颈不同。